

# Module 34: Design Patterns in C++

*Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++*

---

## Module Overview

---

This module gives students a catalogue of ten proven designs and, more importantly, the vocabulary to discuss them. Each pattern is presented as a problem, a solution shape, and a runnable example: creational (Factory, Builder, Singleton - with honest warnings), structural (Adapter, Composite, Decorator), and behavioural (Strategy, Observer, State, Command). A brief look at MVC shows patterns combining into an architecture, and a selection table ties each pattern to the situation it solves. It is the largest module in Part 3 and is designed to run over two sessions.

**Suggested Duration:** 2 sessions of 75–90 minutes (creational and structural; then behavioural)

## Learning Objectives

---

By the end of this module, students will be able to:

- Explain what a design pattern is and why shared pattern vocabulary matters.
- Implement Factory and Builder, and explain the risks of Singleton and its usual alternative.
- Implement Adapter, Composite and Decorator, and distinguish Adapter from Decorator.
- Implement Strategy, Observer, State and Command, and distinguish Strategy from State.
- Describe Model-View-Controller as a combination of patterns.
- Choose a pattern from the problem it solves - and decide when not to use one.

## Prerequisites

---

- Module 33 (SOLID) and Module 29 (interfaces).
- Module 26 (ownership) - observers and composites depend on clear ownership.

## Materials & Instructor Prep

---

- 34-design-patterns/examples/ 01 to 10 - one per pattern.
- The 'Choosing a pattern' table from the README as a handout students can keep.

## Key Vocabulary

---

| Term           | Definition                                                       |
|----------------|------------------------------------------------------------------|
| Design pattern | A named, reusable solution shape for a recurring design problem. |

---

| Term                | Definition                                                                                                             |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| Factory             | One place that decides which concrete class to create from runtime data, returning a base-class pointer.               |
| Builder             | An object that constructs a complicated object step by step with named, chainable settings.                            |
| Adapter / Decorator | Adapter changes an interface to fit; Decorator keeps the same interface and adds behaviour by wrapping.                |
| Composite           | Leaves and groups share one interface so a whole tree can be treated like a single item.                               |
| Strategy / State    | Swappable behaviour behind an interface; a strategy is usually chosen from outside, a state switches itself on events. |
| Observer            | A subject notifies subscribed observers of events without knowing what they do.                                        |
| Command             | A request turned into an object with execute() and undo(), so it can be queued, logged or undone.                      |

## Lesson Outline & Instructor Talking Points

### Session 1 - 1. What patterns are (10 min)

- Introduce the Gang of Four families and stress vocabulary: 'make the alarm an Observer' communicates a whole structure in four words.

### Session 1 - 2. Creational: Factory, Builder, Singleton (35 min)

- Run examples/01\_factory.cpp: configuration lines become objects, and an unknown type is reported. Show the registry variant.
- Run examples/02\_builder.cpp and compare it with a ten-argument constructor call.
- Run examples/03\_singleton.cpp, then argue for the explicit-dependency alternative shown in the same file.

**Instructor note:** Be candid about Singleton: students will meet it, so they must recognise it, but it is a global variable in disguise and makes testing hard.

### Session 1 - 3. Structural: Adapter, Composite, Decorator (35 min)

- Run examples/04\_adapter.cpp: the supplier's probe fits our interface without either side changing.
- Run examples/05\_composite.cpp: one dailyKwh() question asked of an appliance, a room, or the whole house.
- Run examples/06\_decorator.cpp and contrast it with Adapter (same interface vs different interface).

### Session 2 - 4. Behavioural: Strategy and Observer (35 min)

- Run examples/07\_strategy.cpp: letter grades and WASSCE grades on the same report card.
- Run examples/08\_observer.cpp. Emphasise the lifetime rule: observers must unsubscribe before they are destroyed.

## Session 2 - 5. Behavioural: State and Command; MVC (35 min)

- Run examples/09\_state.cpp and draw the pump's state diagram on the board alongside the code.
- Run examples/10\_command.cpp: run three commands and undo them all.
- Close with MVC and the selection table, and the warning that patterns are tools, not goals.

**Instructor note:** Ask students, for each pattern, 'what change does this make easy?' - the answer is the pattern's real justification.

## Common Student Misconceptions

| Misconception                                     | How to Address It                                                                                                                     |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| "Good design uses as many patterns as possible."  | Patterns add structure that must be justified by a real problem. A pattern in every class is as hard to read as none.                 |
| "Singleton is the clean way to share an object."  | It hides dependencies and blocks testing. Prefer creating one object in main and passing it to those who need it.                     |
| "Strategy and State are the same pattern."        | Both swap an object behind an interface, but strategies are chosen from outside while states change themselves in response to events. |
| "Observers can be destroyed whenever convenient." | The subject keeps raw pointers to them; an observer destroyed without unsubscribing leaves the subject with a dangling pointer.       |

## Exercises & Module Project

Exercise: 34-design-patterns/exercises/exercise1.cpp - an automatic school bell combining Strategy (normal vs exam-day schedules), Observer (classroom speakers, staff alerts, a log) and Factory (listeners built from configuration lines, including an invalid one). Project (three tracks): Generic - a school events hub; EE - a smart-home controller; Biomedical - a ward monitoring platform. Each implements at least five patterns including Factory, Strategy and Observer, runs a scripted scenario, and documents every pattern (and one deliberate non-use) in PATTERNS.md.

**Facilitation guidance:** Mark PATTERNS.md as heavily as the code: each section should name the real problem, the roles, and the change the pattern makes easy. Check that observers unsubscribe, ownership uses `unique_ptr`, and the scenario trace makes every pattern's effect visible.

## Discussion & Comprehension Check Questions

### 1. What is the difference between Adapter and Decorator?

*Guidance:* Adapter converts one interface into another that callers expect; Decorator keeps the same interface and adds behaviour around the wrapped object.

### 2. Why must an observer unsubscribe before it is destroyed?

*Guidance:* The subject stores a non-owning pointer to it; notifying a destroyed observer uses a dangling pointer.

### 3. What problem does Factory solve that a constructor call doesn't?

*Guidance: It keeps the decision of which concrete class to create in one place, so the rest of the program depends only on the base class and new types need one change, not many.*

### Assessment Ideas

---

- Pattern-matching quiz: ten short scenarios; students name the most suitable pattern for each (or 'none').
- Students draw the text UML of one pattern from their project with each role labelled.

### Answer Key Notes

---

See 34-design-patterns/exercises/solutions/exercise1\_solution.cpp.

### Wrap-Up & Bridge to Next Module

---

Bridge forward: "Our designs are now flexible. But how do we KNOW they work - and keep working as they change? Next module: automated tests, test-driven development, debuggers and sanitizers."

### Instructor Tips

---

- Split the module across two sessions as outlined; trying to cover ten patterns in one sitting leaves none understood.
- If time is short, protect Factory, Strategy and Observer - the three every project and the capstone require.